

Data-Driven Threshold Scheduling for LLM Inference: Preventing Memory Eviction Cascades

Research Problem

Large Language Models (LLMs) power chatbots, search engines, and coding assistants, serving millions of queries daily. Efficient inference scheduling is critical: ChatGPT alone incurs daily costs exceeding \$700,000. Unlike traditional batch processing, LLM inference faces a unique challenge: the Key-Value (KV) cache grows dynamically during response generation, creating resource demands that change unpredictably during processing.

The KV cache stores attention keys and values to avoid quadratic recomputation, but creates severe capacity challenges. When memory is exceeded, systems must evict prompts and restart them from scratch—triggering **eviction cascades** that degrade throughput by 20–30%. Critically, this failure occurs even when memory capacity exceeds theoretical requirements: a system that *should* be stable becomes unstable under naive scheduling. Classical OR approaches (Shortest Job First, FCFS) assume fixed job sizes and known processing times, making them inapplicable when KV cache grows dynamically and output lengths are unknown at arrival.

We address two questions: (1) How should we schedule LLM inference to prevent eviction cascades? (2) How do we handle unknown output lengths without prediction?

Methodology

We formulate LLM inference as a multi-stage online scheduling problem under memory constraints. Prompts arrive stochastically with known input lengths but unknown output lengths. Each prompt progresses through prefill (processing input) and decode (generating output token-by-token) phases, with KV cache memory growing linearly with output length.

Fluid Dynamics Benchmark. We model iteration time as $\tau = d_0 + d_1 M$, where M is total KV cache memory and d_0, d_1 capture compute and memory bandwidth costs. Using fluid dynamics from queueing theory, we derive equilibrium conditions where arrival rates match completion rates, establishing a tractable benchmark $\text{Throughput}^* = \sum_{j=1}^m \lambda_j l'_j$ (total tokens generated per unit time).

WAIT Algorithm (Known Output Lengths). The *Waiting for Accumulated Inference Threshold* algorithm maintains load balance through threshold-based admission. For each prompt type j , WAIT sets threshold n_j derived from fluid equilibrium. Type- j prompts enter a batch only when $n_{j0}^t \geq n_j$ (at least n_j prompts waiting). This prevents eviction cascades by ensuring arrivals

never exceed completions, keeping the system near equilibrium. The threshold mechanism also *decouples* multi-type dynamics, enabling independent per-type analysis via coupling arguments.

Nested WAIT Algorithm (Unknown Output Lengths). When output lengths are unknown, we exploit a key insight: *prompts classify themselves on-the-fly*. Short prompts complete early; long prompts advance to later segments. Nested WAIT organizes inference into m nested segments, where segment k handles prompts from decode stage $l'_{k-1} + 1$ to l'_k . Each segment operates with its own threshold n_k . Prompts naturally reveal their type by completing or continuing at segment boundaries—no prediction needed.

A **safety buffer** with logarithmic overhead $O(\ln(T/\delta))$ provides high-probability protection against memory overflow. This addresses the fundamental lower bound we establish: when output lengths are unknown, some extra memory beyond fluid equilibrium is necessary (but logarithmic suffices).

Results Summary

Theoretical Guarantees. We prove asymptotic optimality for both algorithms:

- **WAIT:** Throughput gap $\text{Throughput}^* - \mathbb{E}[\text{Throughput}] = O((\zeta T)^{-1/2})$; latency and TTFT $= O((\zeta T)^{1/2})$ in the boundary case, and $O(1)$ with slack.
- **Nested WAIT:** Throughput gap $= O((\zeta T)^{-1})$; latency and TTFT $= O(1)$, with memory overflow probability $\leq \delta$.

We also establish matching lower bounds: (i) FCFS incurs $\Omega(1)$ throughput gap even with sufficient capacity; (ii) any non-predictive policy requires extra memory beyond M^* when output lengths are unknown.

Experimental Validation. Using Microsoft Vidur simulator (NVIDIA A100 GPU) on synthetic and real-world data (LMSYS-Chat-1M, 50,000 prompts), we compare against vLLM and Sarathi (state-of-the-art FCFS-based engines):

- **Throughput:** 12–25% higher than baselines; 40% higher with prefill-decode disaggregation.
- **Latency:** 30% lower end-to-end latency.
- **Stability:** Baselines become unstable (latency $\rightarrow \infty$) at high arrival rates where WAIT remains stable.

Practical Extensions. We develop segment-wise Nested WAIT that groups m types into $L \leq m$ segments, addressing robustness to varying type distributions and long decode lengths where per-stage thresholds are infeasible.

Contribution and Impact

This work bridges service operations theory with AI systems. We identify the core operational challenge in LLM inference—dynamic memory growth causing eviction cascades—and develop

scheduling algorithms with theoretical guarantees. The threshold mechanism provides a principled approach to maintaining load balance, directly applicable to emerging LLM-powered services.

References

- Agrawal, A., et al. (2023). Sarathi: Efficient LLM inference by piggybacking decodes with chunked prefills. *arXiv:2308.16369*.
- Kwon, W., et al. (2023). Efficient memory management for large language model serving with PagedAttention. *SOSP 2023*.
- Zheng, L., et al. (2023). LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. *arXiv:2309.11998*.
- Agrawal, A., et al. (2024). Vidur: A large-scale simulation framework for LLM inference. *MLSys 2024*.
- Zhong, Y., et al. (2024). DistServe: Disaggregating prefill and decoding for goodput-optimized LLM serving. *OSDI 2024*.